



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México

Navegación autónoma para transporte de carga con Puzzlebot

Ivan A. Melo Salgado A0179856

Marco. A. González Fernández A01799122

Marisol S. Ramírez Herrera A01747396

Integración de robótica y sistemas inteligentes

Dr. Alejandro Aceves López

Dr. Miguel Ángel Gálvez Zuñiga

11 de junio 2026

1. Introducción

La robótica móvil autónoma constituye uno de los campos con mayor crecimiento dentro de la ingeniería contemporánea. Tiene como objetivo diseñar sistemas capaces de interactuar con el entorno, procesar información sensorial y tomar decisiones sin intervención humana (Siegwert et al, 2011). Para lograrlo, se emplean algoritmos de navegación, control y técnicas de localización. Esta competencia los convierte en una herramienta crucial para resolver tareas complejas en condiciones cambiantes.

La navegación autónoma es una de las tareas más importantes en la robótica móvil. El sistema debe estimar su posición, identificar obstáculos y generar acciones de control acordes a las condiciones operativas. Para ello, se emplean algoritmos reactivos que permiten al robot bordear los obstáculos, así como técnicas de seguimiento de trayectoria que corrigen desviaciones. El transporte de objetos añade una capa adicional: el robot debe localizar una carga, alinearse frente a ella y manipularla mediante un mecanismo físico. En conjunto, la integración de estas capacidades permite el desarrollo de sistemas robóticos que operan en entornos dinámicos e impredecibles.

El presente trabajo tiene como objetivo implementar un sistema de navegación autónoma capaz de transportar un cubo con el Puzzlebot LiDAR Edition desarrollado por Manchester Robotics. El robot deberá detectar la carga, recogerla con un mecanismo de pala, navegar hacia la estación de destino para descargar, y regresar a su punto de origen.

2. Descripción del reto

El reto consiste en desarrollar un sistema de navegación autónoma para un robot móvil diferencial (Puzzlebot) capaz de realizar una tarea de transporte de carga. Ambas estaciones se encuentran delimitadas mediante marcas cuadradas de 30×30 cm colocadas sobre el piso y separadas por una distancia aproximada de dos metros. El cubo incorpora un marcador ArUco que facilita su identificación. (**Figura 1**).

Figura 1. Visualización del reto final



Nota. Imagen obtenida a través de la plataforma Experiencia Tec21 (Aceves, 2026)

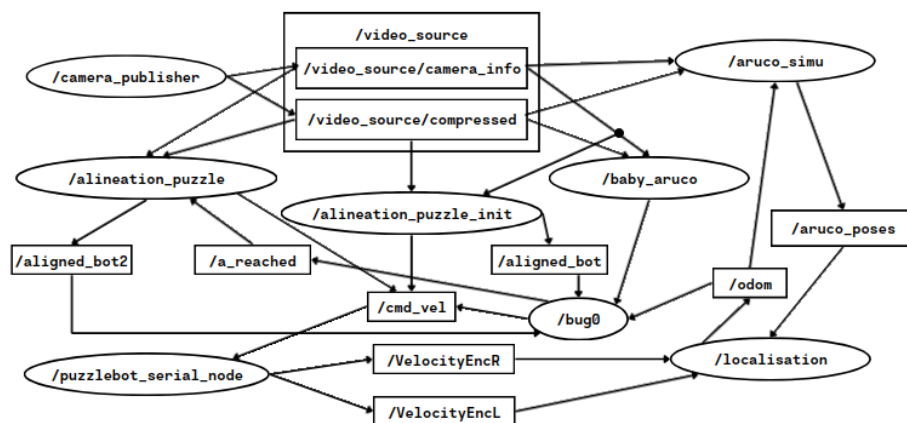
La ejecución inicia con el Puzzlebot en una posición intermedia entre las dos estaciones. El cubo está colocado en la estación de carga. A partir de esta configuración, el sistema debe detectar el ArUco asociado al objeto para estimar su posición relativa. Posteriormente, el robot debe alinearse frente al cubo y efectuar la recolección mediante un mecanismo de pala accionado automáticamente.

Durante el transporte de la carga, el robot debe desplazarse hasta la estación de descarga. En el entorno pueden aparecer obstáculos que interfieran con la ruta prevista. Ante esta situación, el sistema debe detectar la obstrucción y ejecutar una maniobra de evasión. La etapa final consiste en depositar el cubo dentro de la estación de descarga con una posición lo más cercana posible al centro del área delimitada. Tras completar la entrega, el robot debe regresar de manera autónoma a su ubicación inicial.

3. Solución propuesta

La arquitectura propuesta está compuesta de tres subsistemas principales: localización, detección visual de marcadores ArUco y navegación reactiva (Figura 2). La localización se realiza mediante odometría diferencial complementada con un Filtro de Kalman Extendido (EKF), mientras que la percepción visual se basa en la detección de marcadores ArUco. Por su parte, la navegación se implementa mediante el algoritmo reactivo Bug0. La integración de estos componentes permite que el robot complete la tarea de transporte de forma autónoma.

Figura 2. Arquitectura del sistema.



La estimación de la pose del robot se obtiene a partir de las lecturas de los encoders de ambas ruedas. Debido a la acumulación de errores inherentes en la odometría, se emplea un EKF para corregir la estimación utilizando como referencia los marcados ArUco ubicados en posiciones conocidas. Esta fusión de información reduce la incertidumbre, de modo que la localización es más precisa durante el desplazamiento.

La detección de ArUco se implementa mediante la biblioteca OpenCV. Para garantizar una estimación de pose lo más exacta posible, la cámara fue calibrada previamente con un tablero de ajedrez (*checkerboard*). Esto permitió obtener los coeficientes de distorsión necesarios para corregir la imagen. Correspondientemente, se utilizan diccionarios distintos

para diferenciar los marcadores de referencia del entorno (4x4) y el marcador asociado al cubo de carga (5x5).

La navegación autónoma se basa en el algoritmo Bug0. El robot avanza hacia su objetivo; al detectar un obstáculo mediante el sensor LiDAR, ejecuta una maniobra de evasión hasta recuperar una trayectoria libre. Finalmente, la detección, alineación y recolección del cubo se realizan mediante la cámara.

3. 1. Nodos de Alineación

Para lograr que el robot se posicione correctamente frente a los marcadores ARUco, se implementaron dos nodos de alineación que comparten una arquitectura basada en una máquina de estados y se ejecutan en momentos distintos. Ambos nodos procesan imágenes comprimidas provenientes de la cámara, detectan el centroide del marcador objetivo y aplican una estrategia de control proporcional sobre la velocidad del robot.

El nodo *alination_puzzle_init* utiliza el diccionario *DICT_4X4_1000* y busca exclusivamente el ArUco con el ID 0, que corresponde a aquel que se encuentra en la zona de carga. Su máquina de estados está compuesta de tres fases: *searching*, *aligning* y *waiting*. En el estado *searching*, el robot gira a 0.3 rad/s en velocidad angular hasta detectar el marcador deseado. Una vez detectado, cambia al estado *aligning*, donde calcula el error en píxeles entre el centroide del marcador y el centro de la imagen tomando en consideración un offset de 15 píxeles que buscan compensar la posición física de la cámara, aplicando un control proporcional con $k_w = 0.003$ y una saturación de ± 0.3 rad/s.

$$w = -k_w * error_{píxeles}$$

Cuando el error es menor a 10 píxeles, el robot se detiene y se publica *aligned_bot = True* sobre el tópico correspondiente, posteriormente el nodo transita al estado *waiting* donde permanece durante 5 segundos antes de concluir su ejecución.

El nodo *alination_puzzle* utiliza el diccionario *DICT_5X5_1000* y filtra su búsqueda exclusivamente al ArUco con el ID 34, que corresponde al marcador que se encuentra en la caja de recolección. Este nodo extiende la máquina de estados del nodo anterior incluyendo dos fases adicionales: *advancing y pushing*. Su activación está condicionada a la recepción de *a_reached*, garantizando que el robot no inicie la maniobra antes de que el nodo predecesor haya acabado. Una vez activo, el nodo sigue la misma lógica de búsqueda y alineación que el primero, pero al alcanzar el threshold de error este transita al estado *advancing*, donde el robot se desplaza hacia el marcador combinando una estrategia de control angular y velocidad lineal variable

$$v = v_{max} - kv * |error_{pixeles}|$$

El avance continúa hasta que el centroide del marcador alcanza el 90% de la altura del frame, o bien hasta que el marcador se pierde por más de 2 segundos, en cuyo caso se asume que el robot ya está lo suficientemente cerca del marcador. En ambos casos, el nodo transita al estado de *pushing*, donde aplica velocidad lineal constante de 0.04 durante 4.5 segundos. Al finalizar, publica *aligned_bot2 = True* y concluye su ejecución.

Figura 3. Máquina de estados *alination_puzzle_init*

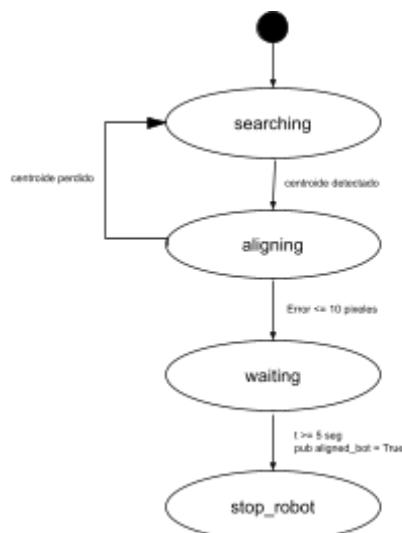
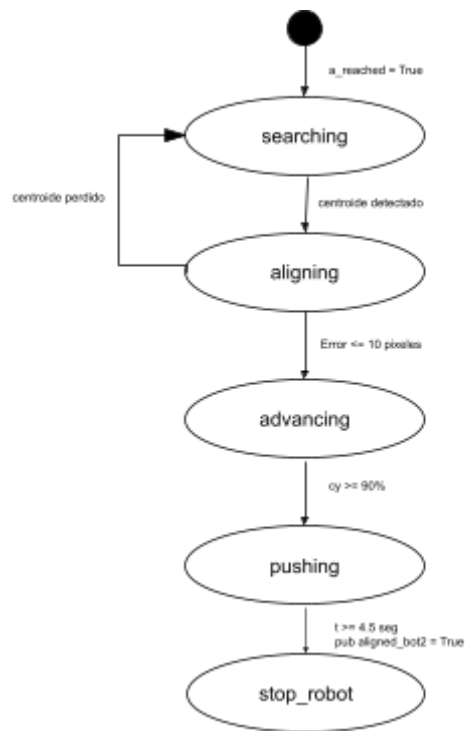


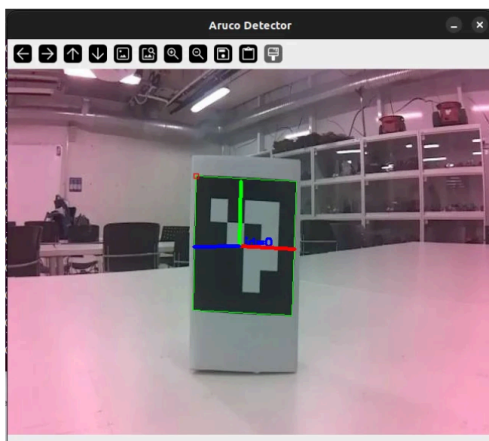
Figura 4. Máquina de estados alineation_puzzle



3.2. Nodos de detección de arUcos

Se implementaron dos nodos de detección de ArUcos cuyo pipeline de procesamiento es estructuralmente idéntico, diferenciándose por el diccionario utilizado, el tamaño del marcador y la forma en la que se expresa la pose resultante en el espacio. Ambos nodos se suscriben a los tópicos de imagen comprimida, información de cámara y odometría y publican las poses estimadas de los marcadores detectados como un mensaje de tipo Pose Array.

Figura 5. Detección de ArUcos



Al recibir cada frame, se obtienen las esquinas y los identificadores de los marcadores presentes en la imagen. Para cada marcador detectado, se resuelve el problema de estimación de pose mediante el algoritmo *solvePnP*, que a partir de la correspondencia entre los puntos del marcador en su marco local y sus proyecciones en el plano de la imagen determina el vector de traslación y el vector de rotación en representación homogénea de Rodrigues. Con estos se construye la transformada homogénea del marcador respecto a la cámara, la cual se combina con la transformada fija de la cámara respecto al robot (T_{rc}), que codifica la posición y orientación de la cámara en el chasis incluyendo la rotación (R_{yz}) que alinea sus ejes.

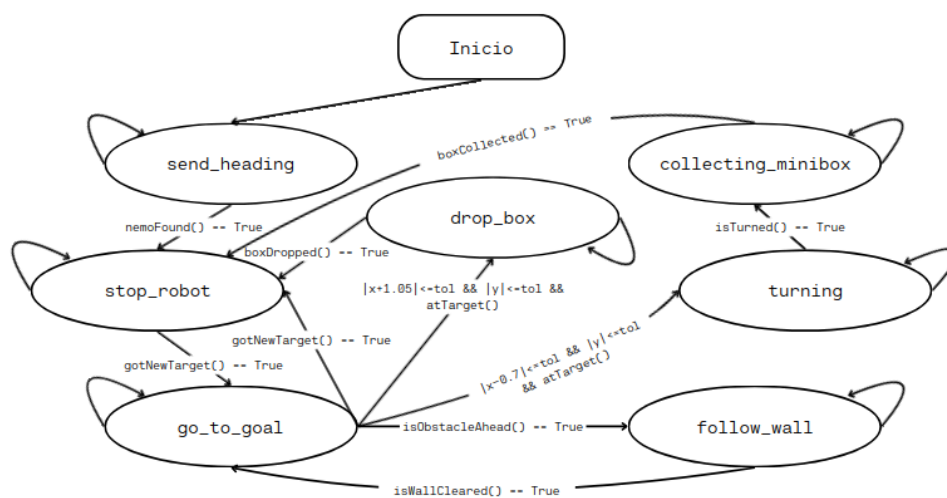
El primer nodo *aruco_simu* utiliza el diccionario *DICT_4X4_100* con ArUcos de 0.1 m y filtra IDS menores a 9. Expresa la pose del marcador en el marco universo combinando la transformada de odometría del robot (T_{ur}), construida en cada callback de odometría a partir de la posición y el ángulo yaw del robot con la transformada del robot a la cámara (T_{rc}) y la transformada de la cámara al ArUco (T_{ca}). Adicionalmente aplica una corrección sobre la posición publicada para compensar el desfase entre el centro del robot y el origen de odometría, publicando el resultado en el tópico *aruco_poses*.

El segundo nodo *baby_aruco* utiliza el diccionario *DICT_5X5_100* con ArUcos de 0.03 m, y filtra IDs menores a 50. A diferencia del primero, la pose del marcador no se expresa en el marco global sino en el marco del robot, por lo que la pose del marcador se expresa directamente en el marco de la cámara mediante (T_{rc}) y (T_{ca}). Esto es adecuado para su caso de uso, ya que opera en proximidad al marcador de entrega y no requiere una estimación de pose global. En ambos nodos, la pose final se convierte a cuaternión y se publica codificando el ID del marcador en el campo *pose.position.z*.

3.3. Nodo Bug 0

El nodo **Bug0** es el núcleo del sistema de navegación del Puzzlebot. Su función es administrar la secuencia completa de la misión, desde la alineación hasta el depósito del cubo. La navegación entre objetivos se realiza mediante un controlador proporcional sobre el error de posición y orientación respecto a la meta. El comportamiento del nodo se estructura como una máquina de estados finita de siete estados (Figura X).

Figura 6. Máquina de estados implementada para Bug0.



A continuación, se describe cada estado implementado:

1. **Send heading:** El estado corresponde a la inicialización del sistema. El nodo identifica los marcadores ArUco de referencia presentes en el entorno a través del tópico `/aruco_poses`. Con base en su identificador, publica la orientación inicial en `/heading`. Este valor es utilizado por el EKF para establecer la referencia angular global antes de que comience la navegación.
2. **Stop robot:** El estado **stop robot** actúa como espera entre etapas. El robot permanece detenido publicando velocidades nulas en `/cmd_vel` hasta recibir un nuevo objetivo de navegación.

3. **Go to goal:** El estado **go to goal** implementa la navegación punto a punto mediante el controlador proporcional. La información de pose proviene de */odom*. Si durante la navegación el sensor LiDAR reporta un obstáculo en el sector frontal a una distancia inferior a 0.3 m, el sistema transita al estado **follow wall**.
4. **Follow wall:** El estado **follow wall** implementa la estrategia reactiva de evasión propia de Bug0. Las lecturas del LiDAR se dividen en 6 sectores para estimar la proximidad del obstáculo: frontal, frontal-derecho, frontal-izquierdo, derecha e izquierda. El robot sigue el contorno del objeto manteniendo una distancia de seguridad de 0.25 m. La transición de regreso a **go to goal** ocurre cuando el camino se encuentra despejado.
5. **Turning:** El estado **turning** ejecuta una rotación sobre el eje vertical del robot hasta alcanzar una orientación $|\theta| < 0.1$. Su objetivo es garantizar que la aproximación posterior al cubo se realice desde una configuración adecuada para el mecanismo de carga.
6. **Collecting minibox:** El estado **collecting minibox** gestiona la alineación y recolección del cubo a partir de la información publicada en */mini_aruco_pose*. Se realizan proyecciones trigonométricas sobre la distancia y orientación estimadas del marcador ArUco para calcular puntos intermedios de alineación. El robot corrige primero su orientación y posteriormente ajusta su posición lateral (*a*). Una vez alcanzada la configuración deseada, acciona la pala mediante */ServoAngle*, publica la señal de inicio en */a_reached* para activar el modo de alineación fina y espera la confirmación en */aligned_bot2*. Al recibirla, sube la pala y retrocede para asegurar la carga, estableciendo como nuevo objetivo la estación de descarga en $(-1.05, 0.0)$.
7. **Drop box:** El estado **drop box** ejecuta el depósito de la carga en la estación de descarga. El mecanismo de pala libera el cubo mientras el robot retrocede una distancia fija. Posteriormente, se establece como objetivo el retorno a la posición inicial $(-0.0, 0.0)$, lo que marca la conclusión de la misión.

3.4. Nodo de Localización

El nodo **Localisation** implementa la estimación de pose del robot mediante odometría diferencial y observaciones visuales de marcadores ArUco. La odometría proporciona una estimación continua, mientras que los ArUco de posición global conocida permiten corregir la deriva acumulada. La combinación de ambas fuentes se lleva a cabo mediante un Filtro de Kalman Extendido (EKF).

Odometría diferencial

El modelo cinemático del Puzzlebot corresponde al de un robot de tracción diferencial. La posición y orientación del robot en el plano se actualizan a partir de las velocidades angulares de las ruedas derecha e izquierda (ω_R, ω_L) medidas por los encoders.

La velocidad lineal y angular del robot se obtienen como:

$$v = \frac{(\omega_R + \omega_L) R}{2}, \quad \omega = \frac{(\omega_R - \omega_L) R}{L}$$

donde $R = 0.0473$ m es el radio de las ruedas y $L = 0.182$ m es la distancia entre ellas.

A partir de estas velocidades, la pose del robot se actualiza en cada intervalo de tiempo mediante:

$$\begin{aligned} x_{k+1} &= x_k + v \Delta t \cos(\theta_k) \\ y_{k+1} &= y_k + v \Delta t \sin(\theta_k) \\ \theta_{k+1} &= \theta_k + \omega \Delta t \end{aligned}$$

La odometría pura acumula error con el tiempo. Para mitigarlo, se incorpora un EKF que corrige el estado estimado cada vez que se detecta un marcador ArUco de posición conocida.

Filtro de Kalman Extendido (EKF)

Con el fin de mejorar la estimación de pose, se implementó un Filtro de Kalman Extendido (EKF). Este algoritmo permite fusionar mediciones provenientes de dos fuentes de

naturaleza distinta: la odometría diferencial y las observaciones de marcadores ArUco detectados.

El funcionamiento del EKF se divide en dos etapas: predicción y corrección. En la etapa de predicción, el filtro utiliza el modelo cinemático del robot y las velocidades angulares de las ruedas para estimar la nueva pose (x, y, θ) . Simultáneamente, se propaga la matriz de covarianza del error mediante la linealización del modelo alrededor del estado actual, empleando las matrices Jacobianas correspondientes:

$$\mathbf{E}_k = \mathbf{H}_k \mathbf{E}_{k-1} \mathbf{H}_k^\top + \mathbf{F}_k \mathbf{L}_k \mathbf{F}_k^\top$$

donde H_k es el Jacobiano del modelo de movimiento respecto al estado, F_k es el Jacobiano respecto al ruido de proceso y L_k es la matriz de covarianza del ruido de los encoders. Este proceso permite cuantificar cómo crece la incertidumbre como consecuencia del movimiento y de los errores introducidos por los encoders.

Cuando se dispone de una o más observaciones de marcadores ArUco, se ejecuta la etapa de corrección una vez por cada marcador presente en el campo de visión del robot. En esta fase se calcula la ganancia de Kalman:

$$\mathbf{K}_k = \mathbf{E}_k \mathbf{G}_k^\top (\mathbf{G}_k \mathbf{E}_k \mathbf{G}_k^\top + \mathbf{R}_k)^{-1}$$

donde G_k es el Jacobiano de la función de observación y R_k es la matriz de covarianza del ruido de medición. La ganancia de Kalman determina el peso relativo entre la predicción del modelo y la medición visual. Finalmente, el vector de estado y la matriz de covarianza se actualizan mediante:

$$\boldsymbol{\mu}_k \leftarrow \boldsymbol{\mu}_k + \mathbf{K}_k (\mathbf{Z}_k - \mathbf{g})$$

$$\mathbf{E}_k \leftarrow \mathbf{E}_k - \mathbf{K}_k \mathbf{G}_k \mathbf{E}_k$$

donde Z_k es la observación real y g es la observación predicha por el modelo. De este modo, el EKF reduce la acumulación de error inherente a la odometría y proporciona una estimación de pose con mayor exactitud.

Para calcular las matrices de covarianza L_k y R_k se aproximó con una linearización de covarianzas medidas a través de experimentos con el hardware en específico. Para estimar la incertidumbre asociada al modelo cinemático, se realizaron experimentos con el robot operando sobre el suelo a diferentes velocidades angulares. En cada condición se registraron muestras de la velocidad medida de ambas ruedas, posteriormente se calcularon las matrices de covarianza correspondientes. A partir de estos datos se obtuvo la relación entre la velocidad angular y su covarianza. Por último, se realizó una aproximación lineal dando como resultado L_k .

Para la incertidumbre de las mediciones de los marcadores ArUco, el robot se mantuvo inmóvil mientras se colocó un marcador a distancias conocidas y con diferentes orientaciones respecto a la cámara. Para cada distancia se registraron múltiples mediciones de posición obtenidas y se calculó la covarianza de las mediciones. Finalmente, se realizó una aproximación lineal que relaciona la distancia entre el robot y el marcador con la covarianza, dando como resultado R_k .

$$L_k = \begin{bmatrix} 0.0173|w_R| - 0.0314 & 0 \\ 0 & 0.0143|w_L| - 0.0124 \end{bmatrix} \quad R_k = \begin{bmatrix} 0.0336d + 0.0366 & 0 \\ 0 & 0.0336d + 0.0366 \end{bmatrix}$$

4. Experimentos para medir el desempeño

La evaluación del sistema se estructuró en fases progresivas, diseñadas para validar cada subsistema. La primera fase consistió en la validación del algoritmo de navegación Bug0 en un entorno simulado mediante Gazebo. Se definió una secuencia de ocho *waypoints* distribuidos en una arena virtual. Se midió el error euclidiano entre la posición objetivo y la posición real alcanzada por el robot en cada punto. La Tabla 1 presenta los resultados de navegación obtenidos en la fase de simulación. Para cada *waypoint* se reporta la posición objetivo (x_t, y_t), la posición real alcanzada (x_r, y_r) y el error euclidiano

Tabla 1. Desempeño del algoritmo Bug 0 en Gazebo

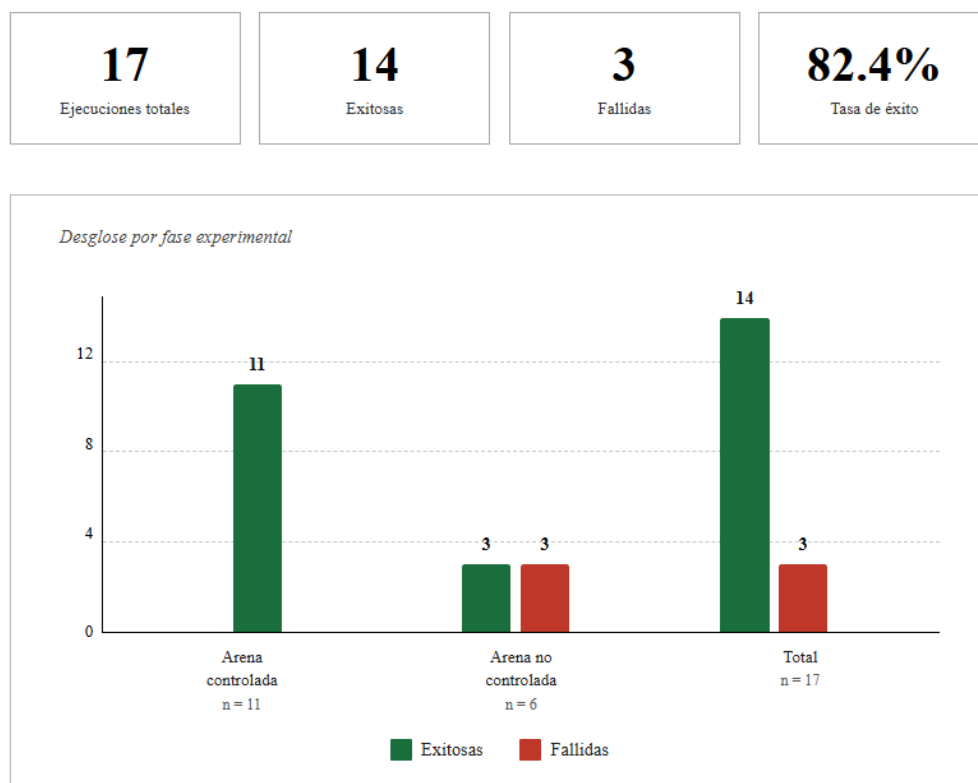
Punto	Posición deseada (x_t, y_t) [m]	Posición estimada (x_r, y_r) [m]	Error [m]
1	(-2.00, 2.00)	(-1.96, 1.94)	0.0722
2	(0.00, 3.00)	(-0.03, 2.97)	0.0423
3	(3.00, 2.00)	(2.94, 2.01)	0.0614
4	(2.00, 0.00)	(1.95, 0.05)	0.0715
5	(3.00, -2.00)	(2.97, -1.96)	0.0506
6	(0.00, -3.00)	(0.03, -2.96)	0.0507
7	(-2.00, -2.00)	(-1.97, -1.97)	0.0428
8	(-3.00, 0.00)	(-2.97, 0.03)	0.0420
<i>Error promedio</i>			<i>0.0542</i>

El error euclidiano promedio obtenido fue de $\bar{d} = 0.054\text{m}$, con un valor máximo de 0.072 m y un mínimo de 0.042 m. Estos resultados indican que el controlador proporcional converge de forma consistente a menos de 8 cm del objetivo en todos los waypoints evaluados.

Con el objetivo de evaluar el desempeño del sistema desarrollado, se realizaron diversas pruebas utilizando el escenario del reto mostrado en la Figura 1. Como métrica de

desempeño se estableció el cumplimiento completo del reto, considerándose una prueba exitosa únicamente cuando el robot lograba completar todas las tareas requeridas y satisfacer las condiciones establecidas. En total se llevaron a cabo 17 ejecuciones experimentales, de las cuales 14 fueron completadas exitosamente, esto corresponde a una tasa de éxito del 82.35 %.

Figura 7. Resumen de completitud en la implementación en físico.



5. Discusión sobre las ventajas y limitaciones de la solución elegida

La solución implementada integró nodos interdependientes: localización mediante odometría con EKF, detección visual de marcadores ArUco, alineación y navegación reactiva mediante el algoritmo Bug0. La combinación de estos nodos le permite al PuzzleBot Lidar Edition ejecutar una tarea de transporte de carga de extremo a extremo de forma autónoma, sin intervención humana durante las pruebas.

5.1. Ventajas

La arquitectura modular propuesta representa una de sus mayores fortalezas. Al separar localización, visión computacional y navegación en nodos independientes que se comunican a través de tópicos de Ros2, cada nodo pudo ser desarrollado, probado y ajustado de forma aislada sin afectar al resto del sistema. Facilitando la depuración y escalabilidad de la solución.

El uso de marcadores ArUco como referencias visuales ofrece una fuente de corrección. Su integración con el EKF permite compensar el error acumulado por la odometría, mejorando la estimación de pose durante el desplazamiento. De manera similar, el algoritmo Bug0 resulta adecuado para el entorno del reto debido a su capacidad de reacción ante obstáculos imprevistos sin necesidad de un mapa previo del entorno.

5.2. Limitaciones

La dependencia de los marcadores ArUco tanto para la localización como para la alineación. Ante condiciones ángulos de visión desfavorables, el sistema se ve comprometido y debe operar únicamente con odometría, lo que incrementa el error acumulado con el tiempo.

El algoritmo Bug0, no garantiza la convergencia al objetivo en todos los entornos. En configuraciones con obstáculos cóncavos o laberintos, el robot puede quedar atrapado en ciclos de evasión sin alcanzar la meta.

La estrategia de alineación y empuje del cubo mediante visión computacional es sensible a cambios en la posición inicial del cubo. Pequeñas variaciones en la ubicación del marcador ArUco del cubo pueden resultar en una alineación insuficiente.

6. Conclusión

Se logró implementar exitosamente una solución tanto en simulación como en un entorno real, integrando los algoritmos de navegación Bug0, el Filtro de Kalman Extendido, detección de marcadores ArUco, estimación de posición del robot y evasión de obstáculos usando la tecnología LiDAR. La interacción de estos componentes permitió que el sistema percibiera su entorno, estimara su posición y tomara decisiones de navegación de acorde a su situación, con el objetivo de cumplir con los lineamientos establecidos en el reto planteado. Los resultados obtenidos demostraron que la combinación de estas técnicas es capaz de resolver de manera efectiva una tarea autónoma de complejidad moderada, aprovechando las ventajas y enfrentando las limitaciones propias de cada método previamente discutidas.

Asimismo, las pruebas realizadas evidenciaron que el sistema posee un nivel de desempeño razonable, logrando completar satisfactoriamente la mayoría de los intentos experimentales. Si bien existen fuentes de error asociadas a la incertidumbre de la odometría, las mediciones visuales y las condiciones del entorno, la fusión sensorial implementada permitió reducir significativamente estos efectos. En consecuencia, el trabajo desarrollado constituye una base sólida para la construcción de sistemas robóticos autónomos más avanzados, capaces de ejecutar tareas específicas y predeterminadas, al mismo tiempo que proporciona una plataforma sobre la cual pueden incorporarse futuras mejoras en percepción, localización y planeación de trayectorias, con el fin de completar tareas aún más complejas.

Referencias

Aceves, L. (2026). Integración de robótica y sistemas inteligentes. ITESM-CEM

Martinez, M. (n.d.). TE3003B: Integration of Robotics and Intelligent Systems 2026.

GitHub. Recuperado de

https://github.com/ManchesterRoboticsLtd/TE3002B_Intelligent_Robotics_Implementation_2025

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots* (2nd ed.). MIT Press.